Day 12 - Review Agents

Week 3: Agents, Tokens, Tests and Security

Lab Report: Auditing AI-written code with an AI Reviewer

PR #104: unsafe profile endpoint, Review Agent findings, proof and fixes

Prepared by: Mirza Yasir Abdullah Baig

Role: AI Engineer

GitHub user: mirzayasirabdullahbaig07

Repository: https://github.com/mirzayasirabdullahbaig07/AI-Internship

Date: 10 October 2026

1. Objective and result

Lab (handbook, Day 12): take the output from Day 11, deliberately introduce an unsafe API call, feed it to a Review Agent using the production brief, and observe how bounding the context makes the AI catch the flaw.

What I did: extended the Day 11 `LoginForm` and added a profile endpoint ("PR #104") that contains deliberately planted flaws. I gave the diff and the exact Review-Agent brief to a reviewer, then **proved every finding with real tests** run against a real HTTP server, fixed the code, and re-ran everything.

Item	Result
Findings raised by the Review Agent	7 (3 High, 3 Med, 1 Low)
Findings confirmed by a failing test on the unsafe code	6 failing tests covering F1-F4 (the 5 unhappy-path auth tests already passed)
Security tests on the fixed code	28 of 28 passed
Whole repository (Days 3-12), npm run verify	Prettier clean, TypeScript clean, 7 suites, 59 tests passed, exit code 0
Verified commit SHA	b7e51db (full: b7e51db7e1178e26ecd2e518369553bf01b7f00a)

Test identity: all test cases use your seeded account as the caller - **Mirza Yasir Abdullah Baig** (user id 1, role user, email mirzayasirabdullahbaig07@users.noreply.github.com) - with Ayesha Khan (id 2) and an admin (id 3) as the people the caller must *not* be able to read. Passwords hashes and API keys in the database are fake lab values.

Honest limits (read first)

• **The Review Agent here was Claude applying the brief, not an independent session.** The same assistant also wrote the planted flaws, so independence is limited. The ready-to-paste prompt (`PROMPT_bounded.txt`) is included so you can run it in a fresh LLM session and compare.

• **The bounded vs unbounded comparison was not run.** I do not claim measured results for it (see section 9).

• No GitHub PR #104 exists; the number comes from the brief and is a local lab label. Nothing has been pushed to GitHub and hosted CI was not run.

2. Starting point and the deliberately unsafe change

Day 11 output (`day-11-task-agent/LoginForm.tsx`): client-side email validation that blocks submit without '@' and shows a red inline error, with one Jest failure-state test.

PR #104 candidate (commit `454e0ec`) changed the form so that after validation it loads a profile, and added a small Node API. Planted flaws:

- The raw `userId` query value is concatenated into the SQL string (SQL injection).
- The caller is authenticated, but the client decides *whose* profile is returned (authorization bypass / IDOR).
- `SELECT *` returns every column, including `password_hash` and `api_key` (data leakage).
- Raw database error messages are returned to the client.
- The only test shipped is a happy-path test that passes even while everything leaks.

The unsafe lines (as text)

```
const userId = url.searchParams.get("userId");                    // line 18: client-chosen id
const rows = db.prepare("SELECT * FROM users WHERE id = " + userId).all(); // line 20
res.end(JSON.stringify({ error: err.message }));                       // line 25: leaks SQL error
```

Size of the PR the reviewer received (git diff --stat main..PR)

```
 day-11-task-agent/LoginForm.tsx                 | 18 ++++++++--
 day-12-review-agents/src/auth.js                | 15 ++++++++
 day-12-review-agents/src/db.js                  | 44 ++++++++++++++++++++++++
 day-12-review-agents/src/profileRoutes.js       | 30 +++++++++++++++++
 day-12-review-agents/src/server.js              |  9 +++++
 day-12-review-agents/tests/profile.happy.test.js | 22 ++++++++++++
 6 files changed, 135 insertions(+), 3 deletions(-)
```

Reachability check: the endpoint is reachable at `GET /api/profile` through `server.js` and the form calls it, so the flaws are exploitable, not dead code.

3. The Review-Agent brief (verbatim)

This is the exact brief from the handbook. It bounds the reviewer's role, input, risk vectors, output format and what it must *not* do.

```
System role: You are a strict, adversarial Security & QA Reviewer.
Input: A proposed Git diff for PR #104.
Task: Evaluate the diff strictly against the following risk vectors:
  - Authorization bypass (Are IDs trusted from the client?)
  - Data leakage (Are unnecessary fields returned in the JSON?)
  - Missing tests for unhappy paths.
Output format:
  - Risk Level (High/Med/Low)
  - Finding (Line number and description)
  - Required Fix.
Do NOT suggest style or formatting changes. Focus on logic and security.
```

Input given to the reviewer: the 194-line diff of PR #104 (file `review-input/PR-104.diff`), plus one line of context that the PR was only approved for the Day 11 email-validation task.

Core Review Checklist (handbook) applied to this PR

Checklist question	Answer for PR #104
Is the feature reachable in the UI/API?	Yes: GET /api/profile is served and LoginForm calls it, so the flaws are live.
Does the diff exceed the approved scope?	Yes: approved scope was LoginForm.tsx only; the PR adds 5 more files (finding F6).
Are failures new or on the baseline?	All findings are new; the baseline (main) has no profile endpoint.
Are permissions verified server-side?	No: the caller is authenticated but never compared with the requested id (F2).
Does the test prove the claimed behaviour?	No: it only asserts status 200 and a non-empty array, so it passes while leaking everything (F5).

4. Review Agent output

Output follows the required format (Risk Level, Finding with line number, Required Fix) and contains no style or formatting advice. Full text: `review-output/REVIEW_AGENT_OUTPUT.md`.

ID	Risk	Location (commit 454e0ec)	Finding	Required fix
F1	High	profileRoutes.js:20 (value read at :18)	SQL injection: raw userId concatenated into SQL. userId=1 OR 1=1 returns every row; UNION can read any column.	Parameterized query; validate id as a bounded positive integer; reject with 400.
F2	High	profileRoutes.js:12 vs :18; LoginForm.tsx:21	Authorization bypass (IDOR): caller is authenticated but never compared with the requested id; the browser chooses whose row to read.	Derive the target from the session; enforce self-or-admin server-side; 403 otherwise.
F3	High	profileRoutes.js:20 and :22; LoginForm.tsx:25	Data leakage: SELECT * serializes password_hash and api_key; the form only needs name.	Explicit column allow-list and a response object; test that secrets are absent.
F4	Med	profileRoutes.js:25	Internal error disclosure: err.message returned to the client reveals SQL/engine details.	Generic 500 body; log details server-side only.
F5	Med	tests/profile.happy.test.js:19-22	Missing unhappy-path tests: only status 200 and length > 0 are asserted; no 401/403/400/404, injection or leakage tests.	Add tests for auth failures, other users, injection, leakage, error text, invalid and unknown ids.
F6	Med	Whole diff (6 files)	Scope creep: approved scope was LoginForm.tsx only; PR adds auth.js, db.js, profileRoutes.js, server.js and a test; the form now makes a network call.	Split the server work into its own PR with its own brief, owner and review.
F7	Low	LoginForm.tsx:21-25	No res.ok check and no try/catch: a network failure is an unhandled rejection; validation accepts a lone '@' and is client-side only.	Check res.ok, show an error state, tighten the pattern, re-validate on the server when a login endpoint exists.

5. Proof: real tests reproduce the findings

A reviewer can be wrong, so each finding was turned into an automated test that sends real HTTP requests to a real server (in-memory SQLite, fresh data per run). The *same* test file runs against both implementations: `TARGET=vulnerable` is the PR as proposed, the default is the fixed code.

Finding	Test(s)	Vulnerable PR	Fixed code
F1 SQL injection	tautology 1 OR 1=1; UNION SELECT of admin row	FAIL, FAIL	PASS, PASS
F2 IDOR	ask for Ayesha (id 2); ask for admin (id 3)	FAIL, FAIL	PASS, PASS
F3 Data leakage	own profile must not contain password_hash / api_key	FAIL	PASS
F4 Error disclosure	malformed id 1' must not return SQL error text	FAIL	PASS
F5 Unhappy paths	no token, bogus token, __proto__ token, unknown route (all 401/404)	PASS x4 (already handled)	PASS x4
F5 (new, fixed API)	17 tests: self, other user 403, no enumeration, admin 200 without secrets, 9 invalid ids 400, 401, 405	skipped (route does not exist)	PASS x17
F7 LoginForm	5 component tests: bearer token and no user id, invalid email never hits network, lone '@', 401 shows error, network failure	n/a	PASS x5
F6 Scope creep	process finding (diff review), not testable by a unit test	n/a	n/a

Real run against the vulnerable PR (text output; failures are the proof)

```
$ npm run test:vulnerable   (TARGET=vulnerable - expected to FAIL: failures prove the findings)
FAIL day-12-review-agents/tests/security.test.js
  PR #104 security tests (target: vulnerable)
    [PASS] happy path: signed-in user gets their own name (25 ms)
    [FAIL] F2: client-supplied userId of another user (Ayesha) never returns her data (7 ms)
    [FAIL] F2: asking for the admin's id never returns admin data (9 ms)
    [FAIL] F1: tautology injection (1 OR 1=1) does not dump other users (4 ms)
    [FAIL] F1: UNION injection cannot read the admin's secrets (7 ms)
    [FAIL] F3: own profile never exposes password_hash or api_key (4 ms)
    [FAIL] F4: a malformed id does not leak database error text (10 ms)
    [PASS] F5: no token -> 401 (2 ms)
    [PASS] F5: bogus token -> 401 (7 ms)
    [PASS] F5: prototype-key token -> 401 (6 ms)
    [PASS] F5: unknown route -> 404 (5 ms)
  fixed API: GET /api/users/:id (server-side authorization)
Test Suites: 1 failed, 1 total
Tests:       6 failed, 17 skipped, 5 passed, 28 total
```

Result: **6 failed, 5 passed, 17 skipped**. The 6 failures are exactly F1-F4.

A test bug I caught in my own work

My first run showed 7 failures, not 6: the "no token" case wrongly failed. The cause was a test helper bug (a default parameter silently replaced my `undefined` token with a real one), not a vulnerability. I fixed the helper and re-ran. I mention it because the handbook's Rule 5 forbids hiding a failure or weakening an assertion; the assertion was kept, the helper was corrected.

6. Real HTTP responses (text captured from the running servers)

Produced by `node day-12-review-agents/tools/show_attacks.js`. The caller is always Mirza Yasir Abdullah Baig (user id 1, role user). Bodies are shortened here; the full text is in `evidence/attack_responses.txt`.

Attack (caller: user id 1)	Vulnerable PR #104	Fixed code

Own profile	`GET /api/profile?userId=1`
	`HTTP 200`
	`[{"id":1,"name":"Mirza Yasir Abdullah Baig","email":"mirzayasirabdullahbaig07@users.noreply.github.com","role":"user", ...`
		`GET /api/profile`
		`HTTP 200`
		`{"id":1,"name":"Mirza Yasir Abdullah Baig","email":"mirzayasirabdullahbaig07@users.noreply.github.com"}`

IDOR: Ayesha's id	`GET /api/profile?userId=2`
	`HTTP 200`
	`[{"id":2,"name":"Ayesha Khan","email":"ayesha.khan@example.test","role":"user","password_hash":"$2b$10$FAKE-HASH-FOR-L ...`
		`GET /api/users/2`
		`HTTP 403`
		`{"error":"Forbidden"}`

SQLi tautology	`GET /api/profile?userId=1%20OR%201%3D1`
	`HTTP 200`
	`[{"id":1,"name":"Mirza Yasir Abdullah Baig","email":"mirzayasirabdullahbaig07@users.noreply.github.com","role":"user", ...`
		`GET /api/users/1%20OR%201%3D1`
		`HTTP 400`
		`{"error":"Invalid user id"}`

Error probe	`GET /api/profile?userId=1'`
	`HTTP 500`
	`{"error":"unrecognized token: \"'\""}`
		`GET /api/users/1'`
		`HTTP 400`
		`{"error":"Invalid user id"}`

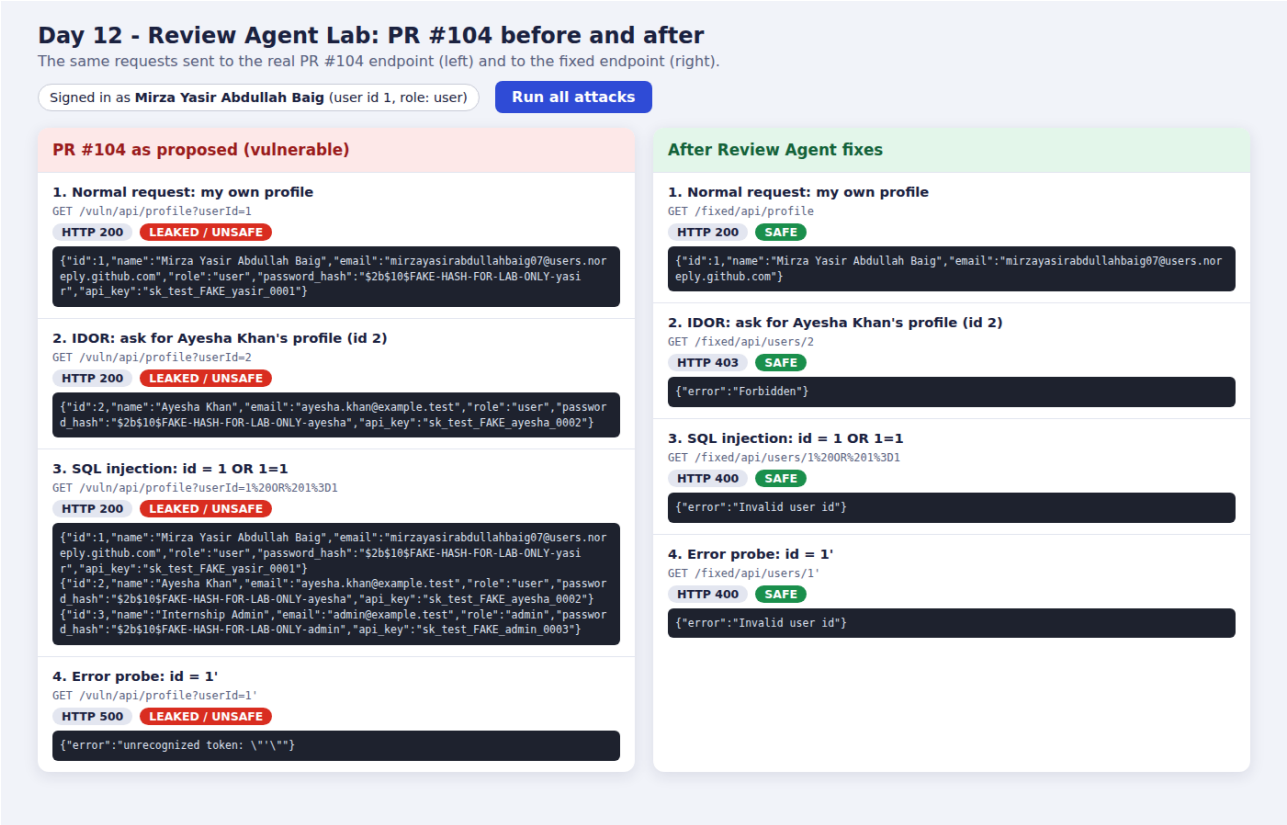
Reading it: on the unsafe PR, a normal user obtains another user's record and fake secrets with one request, dumps all three users with `1 OR 1=1`, and learns the SQL parser's error text. On the fixed code the same intent returns 403 or 400 with generic messages, and the own-profile response contains only id, name and email.

7. Fixes applied

Finding	Change in the fixed code
F1	Prepared statements with bound parameters; id must match ^[1-9]\d{0,8}$ or the request gets 400.
F2	/api/profile means 'me' (identity from the bearer token; any userId is ignored). /api/users/:id allows self or admin only, otherwise 403. A normal user gets 403 even for non-existent ids, so ids cannot be enumerated.
F3	Query selects only id, name, email; response is built field by field. Admins also never receive secrets.
F4	Generic 'Internal server error' body; details logged server-side.
F5	28 security tests plus the updated happy-path test.
F7	LoginForm no longer sends a user id, checks res.ok, handles failures with a red error, and rejects a lone '@'.
F6	Not fixed in code: in a real team this PR should be split. Recorded as a process action.

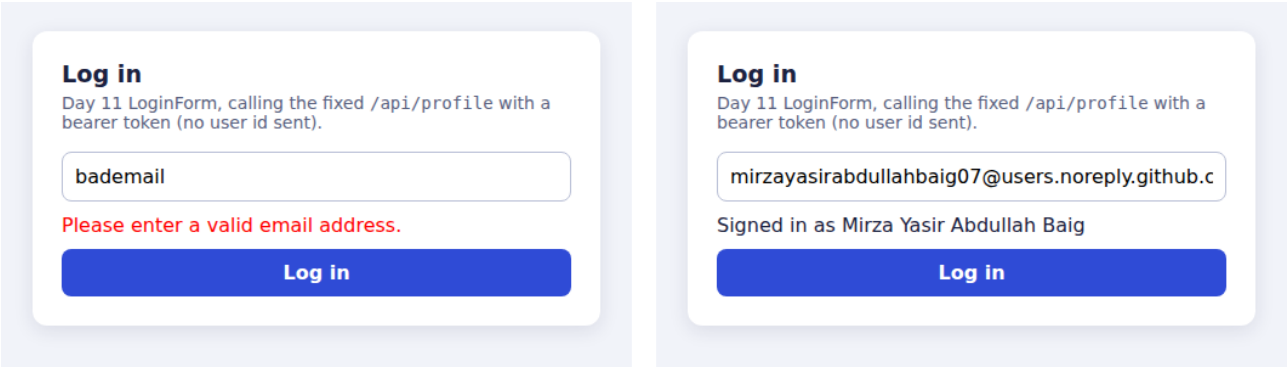
8. Demo screenshots of the working code

Run with `npm run demo:day12` and open `http://localhost:4100`. The page sends the same requests, as Mirza Yasir Abdullah Baig, to the real unsafe endpoint and to the real fixed endpoint, and labels each response.



Screenshot 1: live demo after clicking 'Run all attacks' (left: PR #104 as proposed, right: after the fixes)

The Day 11 LoginForm, running in a browser against the fixed API, which now sends only a bearer token and no user id:



Screenshot 2: LoginForm - left: 'bademail' is blocked with a red inline error; right: a valid email loads the profile and shows 'Signed in as Mirza Yasir Abdullah Baig'

9. Observation: what bounding the context did

What I can state from this run:

• **One finding per risk vector, tied to a line.** The required output format (Risk Level, Finding with line number, Required Fix) made each finding concrete enough to turn into a test, which is why six of them could be proven by execution.

• **The three named vectors map directly to the three High findings** (authorization bypass F2, data leakage F3, and, through the raw id, injection F1), and 'missing tests for unhappy paths' maps to F5.

• **The brief's limits kept the output focused:** no style or formatting remarks appeared, and the input was a 194-line diff rather than the repository.

• **Not everything came from the brief.** F4 (error text), F6 (scope creep, from the handbook's checklist question) and F7 (client-side edge cases) went beyond the three listed vectors, so the brief alone would not have surfaced them.

What I cannot claim: I did not run an unbounded prompt (`PROMPT_unbounded.txt`) in a separate session, so there is no measured comparison of how many flaws a vague prompt would miss. The comparison table in the Day 12 README is left for you to fill in with your own LLM runs.

10. Final verification (text output)

Command: `npm run verify` (Prettier check, tsc --noEmit, Jest for Days 3-12) on commit `b7e51db`.

```
$ npm run verify
> ai-internship@1.0.0 verify
> npm run format:check && npm run typecheck && npm test
> ai-internship@1.0.0 format:check
> prettier --check .
Checking formatting...
All matched files use Prettier code style!
> ai-internship@1.0.0 typecheck
> tsc --noEmit
> ai-internship@1.0.0 test
> jest
PASS day-12-review-agents/tests/security.test.js
PASS day-05-password-strength/passwordStrength.test.tsx
PASS day-12-review-agents/tests/LoginForm.profile.test.tsx
PASS day-04-api-retry/fetchWithRetry.test.ts
PASS day-11-task-agent/LoginForm.test.tsx
PASS day-03-typed-tasks/tasks.test.ts
PASS day-12-review-agents/tests/profile.happy.test.js
Test Suites: 7 passed, 7 total
Tests:       59 passed, 59 total
Snapshots:   0 total
Time:        2.462 s
Ran all test suites.
exit code: 0
```

Fixed-code run of Days 11 and 12 (35 tests):

```
$ npx jest day-11 day-12 --verbose   (fixed code)
PASS day-12-review-agents/tests/security.test.js
  PR #104 security tests (target: fixed)
    [PASS] happy path: signed-in user gets their own name (18 ms)
    [PASS] F2: client-supplied userId of another user (Ayesha) never returns her data (3 ms)
    [PASS] F2: asking for the admin's id never returns admin data (8 ms)
    [PASS] F1: tautology injection (1 OR 1=1) does not dump other users (5 ms)
    [PASS] F1: UNION injection cannot read the admin's secrets (6 ms)
    [PASS] F3: own profile never exposes password_hash or api_key (3 ms)
    [PASS] F4: a malformed id does not leak database error text (6 ms)
    [PASS] F5: no token -> 401 (6 ms)
    [PASS] F5: bogus token -> 401 (12 ms)
    [PASS] F5: prototype-key token -> 401 (2 ms)
    [PASS] F5: unknown route -> 404 (6 ms)
  fixed API: GET /api/users/:id (server-side authorization)
    [PASS] self -> 200 with exactly id, name, email (13 ms)
    [PASS] another user's id -> 403 (6 ms)
    [PASS] a non-existent id as a normal user -> 403, not 404 (no user enumeration) (2 ms)
    [PASS] admin may read another user but still never sees secrets (1 ms)
    [PASS] admin asking for a non-existent id -> 404 (6 ms)
    [PASS] invalid id (letters) -> 400 with a generic message (1 ms)
    [PASS] invalid id (negative) -> 400 with a generic message (6 ms)
    [PASS] invalid id (zero) -> 400 with a generic message (3 ms)
    [PASS] invalid id (decimal) -> 400 with a generic message (6 ms)
    [PASS] invalid id (tautology) -> 400 with a generic message (1 ms)
    [PASS] invalid id (quote) -> 400 with a generic message (6 ms)
    [PASS] invalid id (union) -> 400 with a generic message (2 ms)
    [PASS] invalid id (overflow) -> 400 with a generic message (11 ms)
```

```
    [PASS] invalid id (empty-ish) -> 400 with a generic message (1 ms)
    [PASS] no token on /api/users/:id -> 401 (4 ms)
    [PASS] /api/profile ignores a client-supplied userId and returns the caller (1 ms)
    [PASS] non-GET methods are rejected with 405 (1 ms)
PASS day-12-review-agents/tests/LoginForm.profile.test.tsx
  [PASS] valid email calls /api/profile with the bearer token and NO user id, then greets the user (142 ms)
  [PASS] invalid email never reaches the network (24 ms)
  [PASS] a lone "@" is rejected (39 ms)
  [PASS] a 401 from the API shows an error state instead of a broken greeting (21 ms)
  [PASS] a network failure is handled (no unhandled rejection) (15 ms)
PASS day-11-task-agent/LoginForm.test.tsx
  [PASS] shows red inline error when email lacks '@' (46 ms)
PASS day-12-review-agents/tests/profile.happy.test.js
  [PASS] returns the signed-in user's own profile (3 ms)
Test Suites: 4 passed, 4 total
Tests:       35 passed, 35 total
Snapshots:   0 total
Time:        1.692 s, estimated 2 s
Ran all test suites matching day-11|day-12.
```

Git history (Day 11 and 12 portion)

```
*   b7e51db Merge fix/day-12-review-findings (Day 12 Review Agent lab)
|\
| * 613b9f4 fix(day-12): resolve Review Agent findings F1-F5 and F7; add live demo
| * f280c7f test(day-12): reproduce Review Agent findings against PR #104 (red)
| * 454e0ec feat(api): add profile endpoint for LoginForm (PR #104 candidate)
|/
*   943b53f Merge chore/day-12-tooling
|\
| * b28ee01 chore: add cross-env and esbuild, plus day-12 npm scripts
|/
*   bf57b84 Merge feat/day-11-task-agent
|\
| * b0b6cb9 feat(day-11): add LoginForm email validation (task agent output) with failure-state test
|/
*   f3680d5 Merge docs/final-report
|\
| * a0fd137 docs: add Days 1-10 internship report (PDF), screenshots and evidence
|/
*   949965f Merge docs/readme-index
|\
| * d13afb4 docs: add day index, run and push instructions to README
|/
*   e2b84b1 Merge docs/day-10-ci-failures
|\
| * 35413ea docs(day-10): classify sample CI logs and add honest report
|/
```

To push (run on your machine inside the unzipped repo)

```
git remote add origin https://github.com/mirzayasirabdullahbaig07/AI-Internship.git
git push -u origin main
git push origin --all
```

Not done here: pushing, opening a GitHub PR, and hosted CI. Commits after the verified SHA are documentation and evidence only.

Mirza Yasir Abdullah Baig
AI Engineer | github.com/mirzayasirabdullahbaig07